

# Data Analytics (PETR 6397)

## Artificial Neural Network (ANN): Introduction to deep learning

Dr. Ahmad Sakhaee-Pour

Petroleum Engineering Department

University of Houston

Spring 2025

# Discussed topics

- Artificial neural network (ANN)
- Perceptron
- Multilayer perceptron (MLP)
- Backpropagation
- MLP for regression
- MLP for classification

# Artificial Neural networks (ANNs)

- The biological neurons inspired artificial neural networks in our brains
- But they have evolved and become significantly different from the biological brains
- ANNs are the core of deep learning

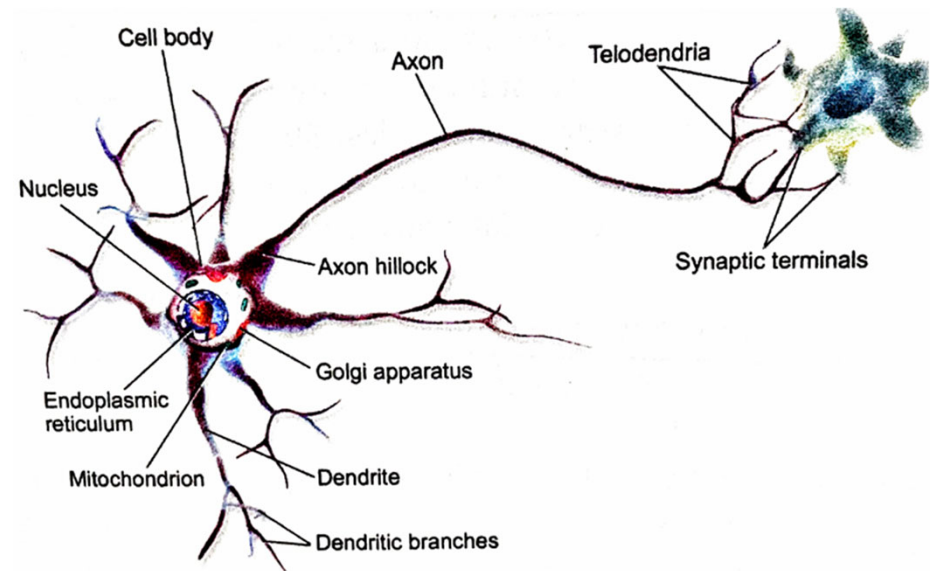


Fig. 10-1

# Long history of ANN with some winters

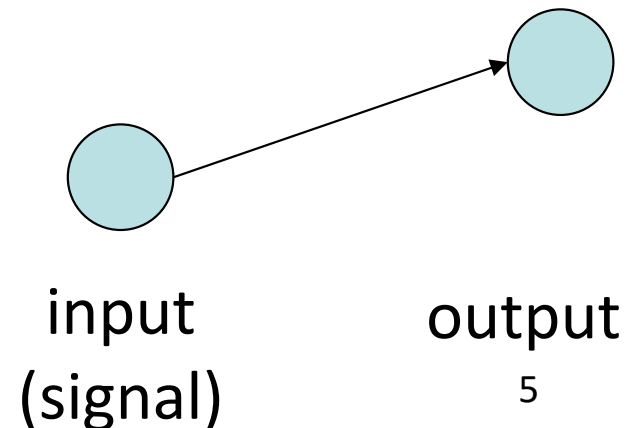
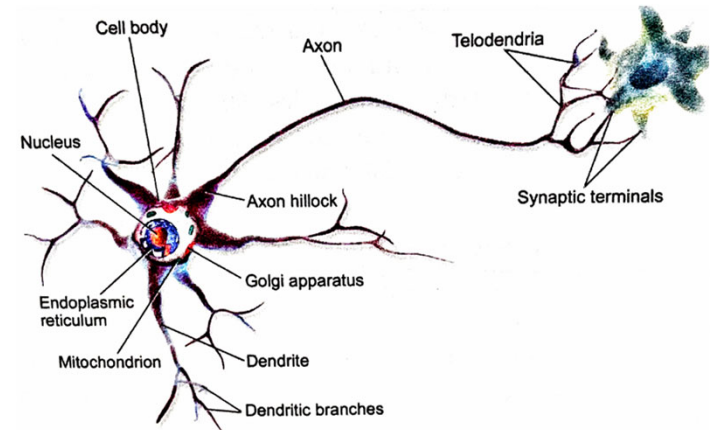
McCulloch introduced ANN in 1943

Will the recent excitement also disappear? Some reasons for hope

1. A vast quantity of data has become available
2. Computing power increased so that we can solve much bigger and more complex problems using neural networks (this is one of the reasons you hear about NVIDIA so often!)
3. Training algorithms have been improved
4. Theoretical limitations of ANNs are not so crucial in practice (we are often happy to find a good solution as opposed to the best solution)

# The main idea of ANN

- We need lots of simple to perform complex computations
- Neurons are often organized in consecutive layers
- Mouse brain has ~ 70 million neurons
- Dolphin brain has ~12 billion neurons
- Human brain has ~ 86 billion neurons



# Logical computations with neuron

- Simplest neuron: McCulloch and Pitts suggested that artificial neuron has some inputs (one or more) and one binary output
- Neuron activates its output when more than a certain number of its inputs are active
- Our artificial neuron networks are far more advanced than the original ones these days

# Perceptron has a simple architecture

- It uses a linear threshold unit (LTU)
- Determines the linear function of inputs

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b \\ = \mathbf{w}^T \mathbf{x} + b$$

$w_1, \dots, w_n$ : weights

$b$ : bias

- Applies a step function

$$h_{\mathbf{w}}(\mathbf{x}) = \text{step function}(\mathbf{x})$$

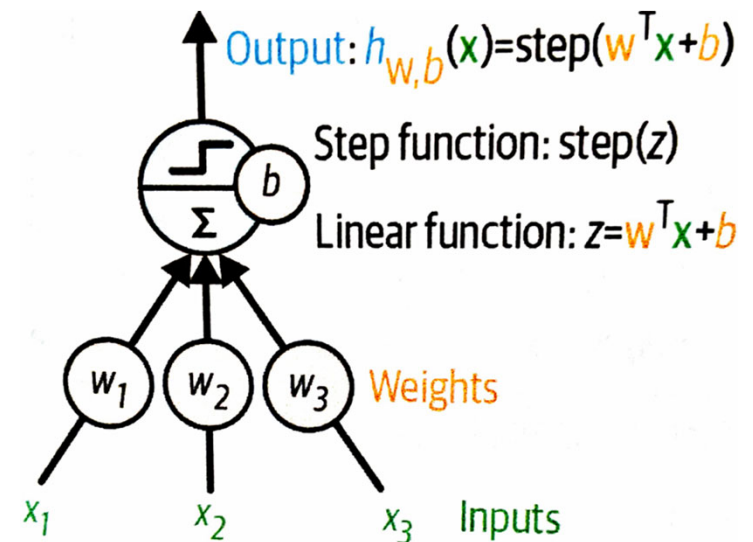


Fig. 10-4

## Run shallow\_net\_1

- Sample calculations to gain a better understanding of how artificial neural network behaves



## Run shallow\_net\_2

- Effects of activation function on artificial neural network

## Run shallow\_net\_3

- Extending the model to include two inputs and two outputs

# Deep neural network (definition)

- Deep neural net contains a deep stack of hidden layers
- We usually consider it deep if it has more than two hidden layers (but there is no universal agreement on this definition)

# Multilayer Perceptron (MLP) and Backpropagation

- Multilayer Perceptron includes one input layer, one or more hidden layers, and one final layer
- Layers close to the input are called lower layers (upper layers are close to the output)
- \*Discussion of backpropagation based on the chain rule
- Run `backprop_chain_rule`

# Backpropagation steps

- It handles one mini-batch at a time
- Each mini-batch enters the network through the inputs (forward pass)
- Forward pass predicts the output and stores intermediate values for the backward pass
- It computes how each bias and weight contributes to the error based on the chain rule
- The algorithm performs a gradient descent to tweak (tune) the weights

# Effects of backpropagation on error

- Basically, backpropagation minimizes error by tuning weights and biases based on the chain rule
- Weights and biases are initialized randomly (with some restrictions that we will discuss later)
- Q: What happens if we set the weights and biases equal to zero?

# MLP

- MLP regression
- MLP classification

# Implementation of MLP regression in a few lines of Python code

```
from sklearn.datasets import fetch_california_housing
...
housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test =
    train_test_split(
        housing.data, housing.target, random_state=42) ①
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, random_state=42)
mlp_reg = MLPRegressor(hidden_layer_sizes=[50, 50, 50], ②
    random_state=42)
pipeline = make_pipeline(StandardScaler(), mlp_reg) ③
pipeline.fit(X_train, y_train) ④
y_pred = pipeline.predict(X_valid) ⑤
rmse = root_mean_squared_error(y_valid, y_pred) ⑥
```



# MLP classification

- MLP can be used for classification too
- Binary classification: We use the Sigmoid activation function after the last output neuron to generate a number between 0 and 1
- Multiclass classification: If we have three or more possible classes, we will have one output neuron per class and the Softmax activation function at the end.

# Softmax function versus sigmoid function

- Softmax function determines the probability of a sample belonging to class  $i$  from  $K$  classes

$$\text{Softmax function } (z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- Sigmoid function decides between two groups:

$$\text{Sigmoid function } (x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}$$

# Numerical example of Softmax function

Outputs before Softmax = [1.1, -0.2, 2.5, 3.6, 5.5]

Probability of input 1: 0.0101

Probability of input 2: 0.0027

Probability of input 3: 0.0409

Probability of input 4: 0.1231

Probability of input 5: 0.8230

Q1: To which group does the sample belong?

Q2: Should the sum of the probabilities be equal to unity?

# Image classification using MLP

- Run MLP\_classification

# Discussion of MLP classification code (slicing)

```
import tensorflow as tf
fashion_mnist =
tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) =
fashion_mnist
X_train, y_train = X_train_full[:-5000],
y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-
5000:]
```

# Scaling down pixel values for better performance

- `X_train, X_valid, X_test = X_train / 255.,  
X_valid / 255., X_test / 255.`

# Sequential design of model (simplest and most intuitive design)

```
tf.keras.backend.clear_session()
tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")])
```

Q1: What does the Flatten layer do?

takes an  $m \times n$  and makes it a  $1 \times (m \times n)$

Q2: How can we decide if this is for classification or regression?

The shape and activation of the output layer

Q3: How would you change it to binary classification?

change to sigmoid activation 1 output node

Q4: How would you change it to regression?

change to ReLU activation w/ 1 output node

# Always run the summary and check the printed values

- `model.summary()`

Model: "sequential"

| Layer (type)      | Output Shape       | Param # |
|-------------------|--------------------|---------|
| flatten (Flatten) | (None, 784)        | 0       |
| dense (Dense)     | (None, 300)        | 235,500 |
| dense_1 (Dense)   | (None, 100)        | 30,100  |
| dense_2 (Dense)   | (None, 10)<br>Text | —1,010  |

Total params: 266,610 (1.02 MB)

Trainable params: 266,610 (1.02 MB)

Non-trainable params: 0 (0.00 B)

- Q1: Where does 30,100 come from?  
All of the intercommunications between Neurons with the current and previous layer
- Q2: Do the output shapes make sense? Why is there none?
- Q3: Are you comfortable with the # trainable parameters?

Because of validation sets it is empirical



# Choosing optimizer (SGD) and loss (cross entropy)

```
model.compile(loss=tf.keras.losses.sparse_categorical_crossentropy,  
optimizer=tf.keras.optimizers.SGD(),  
metrics=[tf.keras.metrics.sparse_categorical_accuracy])
```

# Backpropagation is performed here

```
history = model.fit(X_train, y_train, epochs=30,  
                    validation_data=(X_valid, y_valid))
```

- We use the epoch size of 30
- The loss values are stored in history so we can access them later
- Where does 1719 come from?
- Epoch 1/30 1719/1719 ————— 9s 5ms/step -  
loss: 0.9890...

# Hyperparameters of the model

- How can we improve the model?

Hyperparameter Tuning

- Which parameters can be changed?

batch size, n\_hidden\_layers, n\_neurons, optimizer

- Which parameter would be more important?